

```

In [1]: import pandas as pd
import numpy as np
import random
names = ['Ram', 'Shyam', 'Mohan', 'Raju', 'Harish', 'Dev', 'Raghav', 'Paul', 'Steve', 'Reema', 'Seema',
        'Sita', 'Anjali', 'Priya', 'Amit', 'Travis', 'Diya', 'Michiel', 'Mitchel', 'Darren', 'James',
        'John', 'Sumit', 'Prabhat', 'Vishal', 'Vivek', 'Sabir', 'Shantanu', 'Shivam', 'Sairaj',
        'Khusi', 'Garima', 'Riya', 'Pradeep', 'Pratham', 'Ankit', 'Aditya', 'Vikas']
departments = ['HR', 'SDE1', 'SDE2', 'SDE3', 'Team Lead', 'Jr Manager', 'Peon', 'Management',
              'Sr Manager', 'CTO', 'CFO', 'CEO']
genders = {'Ram': 'Male', 'Shyam': 'Male', 'Mohan': 'Male', 'Raju': 'Male', 'Harish': 'Male',
            'Dev': 'Male', 'Raghav': 'Male', 'Paul': 'Male', 'Steve': 'Male', 'Reema': 'Female',
            'Seema': 'Female', 'Sita': 'Female', 'Anjali': 'Female', 'Priya': 'Female', 'Amit': 'Male',
            'Vikas': 'Male', 'Travis': 'Male', 'Diya': 'Female', 'Michiel': 'Male', 'Mitchel': 'Male',
            'Darren': 'Male',
            'James': 'Male', 'John': 'Male', 'Sumit': 'Male', 'Prabhat': 'Male', 'Vishal': 'Male',
            'Vivek': 'Male', 'Sabir': 'Male', 'Shantanu': 'Male', 'Shivam': 'Male', 'Sairaj': 'Male',
            'Khusi': 'Female', 'Garima': 'Female', 'Riya': 'Female', 'Pradeep': 'Male', 'Pratham': 'Male',
            'Ankit': 'Male', 'Aditya': 'Male', 'Celine': 'Female'}
degrees = ['BSc', 'MSc', 'BTech', 'MTech', 'PhD', 'MBA', 'BCom', 'MCom', 'BBA', 'MBA', 'Engineering']
def gen_employees(num=1000):
    data=[]
    for i in range(num):
        name = random.choice(names)
        gender = genders.get(name, 'Male')
        eid = i+1
        payment = round(random.uniform(40000, 120000),2)
        dept = random.choice(departments)
        address = f'{random.randint(100,999)} {random.choice(['Main St', 'Park Ave', '5th Ave', 'Oak Rd'])}, City'
        degree = random.choice(degrees)
        age = random.randint(20, 60)
        data.append({
            'Name': name,
            'Employee ID': eid,
            'Gender': gender,
            'Payment': payment,
            'Department': dept,
            'Address': address,
            'Degree': degree,
            'Age': age
        })

```

```

    return pd.DataFrame(data)
df = gen_employees()
df.to_csv("Employee.csv")

```

```

In [2]: df = pd.read_csv('Employee.csv')
df.head(2)

```

```

Out[2]:

```

	Unnamed: 0	Name	Employee ID	Gender	Payment	Department	Address	Degree	Age
0	0	Prabhat	1	Male	64909.07	Jr Manager	628 Park Ave, City	BBA	52
1	1	Mitchel	2	Male	87762.10	Management	324 Oak Rd, City	MCom	31

```

In [3]: def mean1(series):
    return sum(series)/len(series)
def median1(series):
    sort = sorted(series)
    n = len(series)
    mid = n//2
    if n%2 == 1:
        return sort[mid]
    else:
        return (sort[mid-1] + sort[mid]) / 2
def mini(series):
    return min(series)
def maxi(series):
    return max(series)
def stdev(series):
    mean = mean1(series)
    variance = sum((x-mean)**2 for x in series) / len(series)
    return variance ** 0.5

```

```

In [4]: summary = df.groupby('Department').agg(
    Age_mean=('Age', mean1),
    Age_median=('Age', median1),
    Age_min=('Age', mini),
    Age_max=('Age', maxi),
    Age_std=('Age', stdev),
    Payment_mean=('Payment', mean1),
    Payment_median=('Payment', median1),
    Payment_min=('Payment', mini),

```

```

    Payment_max=('Payment', maxi),
    Payment_std=('Payment', stdev)
)
print(summary)

```

	Age_mean	Age_median	Age_min	Age_max	Age_std	Payment_mean	\
Department							
CEO	38.506173	38.0	20	60	12.771615	79033.209630	
CFO	43.642857	46.0	21	60	12.581773	78126.160833	
CTO	38.180723	37.0	20	60	11.612358	82715.809759	
HR	42.625000	46.0	20	59	12.800313	78465.213409	
Jr Manager	39.755814	38.0	20	60	12.028006	76936.863488	
Management	36.934066	33.0	20	60	11.832905	80255.129890	
Peon	39.825000	40.0	21	59	9.809148	77761.681000	
SDE1	40.987342	41.0	20	60	12.491510	79026.547089	
SDE2	40.734940	42.0	22	60	10.653451	84553.115783	
SDE3	38.156250	38.5	20	58	11.166662	78170.180156	
Sr Manager	39.356164	41.0	20	60	11.105188	84520.284521	
Team Lead	40.074074	39.5	20	60	11.333908	82095.839074	

	Payment_median	Payment_min	Payment_max	Payment_std
Department				
CEO	80855.520	40767.90	119868.18	21613.059712
CFO	74680.670	40032.40	119938.90	23199.054831
CTO	81360.300	40412.34	119988.26	22899.173922
HR	73452.775	41409.52	119056.03	22635.744120
Jr Manager	75909.995	41767.55	118164.64	20706.208685
Management	82397.390	40377.14	118440.24	23047.968492
Peon	78159.755	40450.53	118375.15	22593.311450
SDE1	80703.430	40864.33	118144.91	22133.229485
SDE2	87600.470	42121.79	119889.35	23390.276602
SDE3	74365.770	41534.90	119764.83	23898.829738
Sr Manager	86565.440	41968.20	119056.04	21243.690571
Team Lead	84676.210	40391.83	119969.77	23947.147606

```

In [5]: df.iloc[2, df.columns.get_loc('Payment')]=2000000
df.iloc[6, df.columns.get_loc('Payment')]=200
df.iloc[10, df.columns.get_loc('Payment')]=1000
df.iloc[70, df.columns.get_loc('Payment')]=2500
df.iloc[700, df.columns.get_loc('Payment')]=6000000

```

```
In [6]: def findoutliers(df,cols):
        outliers = {}
        for col in cols:
            q1= df[col].quantile(0.25)
            q3= df[col].quantile(0.75)
            iqr= q3-q1
            lb=q1-1.5*iqr
            ub=q3+1.5*iqr
            outliers[col] = df[(df[col]<lb) | (df[col]>ub)]
        return outliers
```

```
In [7]: outliers_res = findoutliers(df, ['Age', 'Payment'])
```

```
In [8]: outliers_res['Payment']
```

```
Out[8]:
```

	Unnamed: 0	Name	Employee ID	Gender	Payment	Department	Address	Degree	Age
2	2	Raju	3	Male	2000000.0	SDE2	829 5th Ave, City	MTech	25
6	6	Mitchel	7	Male	200.0	HR	835 Park Ave, City	BCom	23
10	10	Michiel	11	Male	1000.0	SDE3	446 5th Ave, City	MTech	55
700	700	Mitchel	701	Male	6000000.0	Jr Manager	507 Oak Rd, City	MBA	60

```
In [9]: for col, out_df in outliers_res.items():
        print(f"{col}: {len(out_df)} outliers")
```

Age: 0 outliers

Payment: 4 outliers

```
In [13]: # Remove rows where Payment is greater than a realistic threshold (e.g., 200000)
df = df[df['Payment'] < 200000]
```

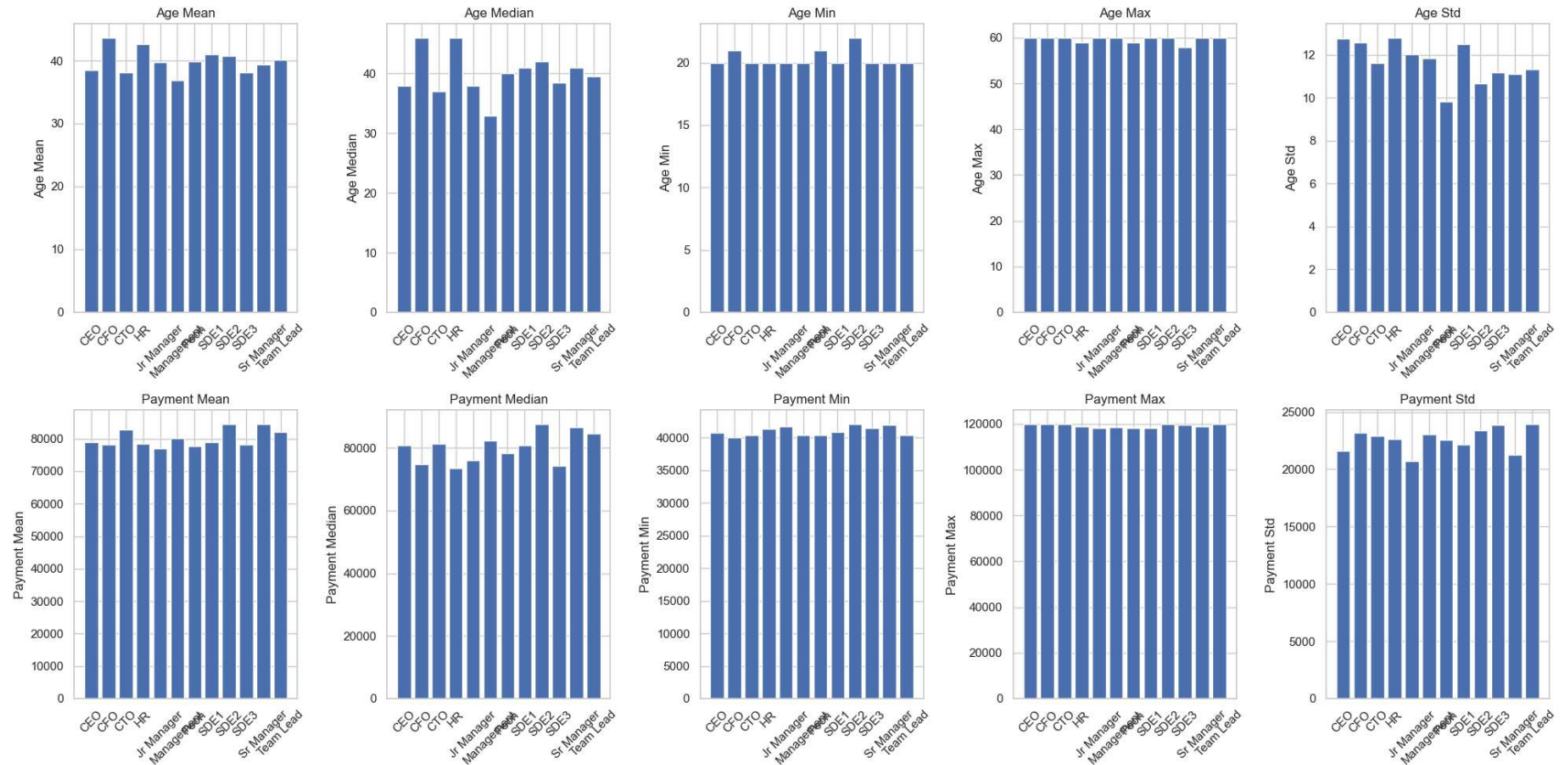
```
In [14]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set(style='whitegrid')
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
sns.boxplot(y=df['Payment'], color='skyblue')
```

```
plt.title('Boxplot of Payment')
plt.subplot(1, 2, 2)
sns.boxplot(y=df['Age'], color='lightgreen')
plt.title('Boxplot of Age')
plt.tight_layout()
plt.show()
```



```
In [15]: fig, ax = plt.subplots(2,5, figsize=(20,10))
ax = ax.flatten()
age = ['Age_mean', 'Age_median', 'Age_min', 'Age_max', 'Age_std']
pay = ['Payment_mean', 'Payment_median', 'Payment_min', 'Payment_max',
      'Payment_std']
for i, stat in enumerate(age):
    ax[i].bar(summary.index, summary[stat])
    ax[i].set_title(stat.replace('_', ' ').title())
    ax[i].set_ylabel(stat.replace('_', ' ').title())
    ax[i].tick_params(axis='x', rotation=45)
for i, stat in enumerate(pay):
    ax[i+5].bar(summary.index, summary[stat])
    ax[i+5].set_title(stat.replace('_', ' ').title())
    ax[i+5].set_ylabel(stat.replace('_', ' ').title())
```

```
ax[i+5].tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.show()
```



```
In [16]: df1 = pd.read_csv('iris.csv')
df1.head(2)
```

```
Out[16]:
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	1	5.1	3.5	1.4	0.2	Iris-setosa
1	2	4.9	3.0	1.4	0.2	Iris-setosa

```
In [17]: df1.describe()
```

Out[17]:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.054000	3.758667	1.198667
std	43.445368	0.828066	0.433594	1.764420	0.763161
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

In [20]: df1.columns

Out[20]: Index(['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm',
'Species'],
dtype='object')

```
In [23]: species_col = 'Species'
species_list = ['Iris-setosa', 'Iris-versicolor', 'Iris-virginica']
for species in species_list:
    print(f"\nStatistical Summary for {species}:\n")
    print(df1[df1[species_col] == species].describe())
```

Statistical Summary for Iris-setosa:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	50.00000	50.00000	50.00000	50.00000	50.00000
mean	25.50000	5.00600	3.41800	1.46400	0.24400
std	14.57738	0.35249	0.38102	0.17351	0.10721
min	1.00000	4.30000	2.30000	1.00000	0.10000
25%	13.25000	4.80000	3.12500	1.40000	0.20000
50%	25.50000	5.00000	3.40000	1.50000	0.20000
75%	37.75000	5.20000	3.67500	1.57500	0.30000
max	50.00000	5.80000	4.40000	1.90000	0.60000

Statistical Summary for Iris-versicolor:

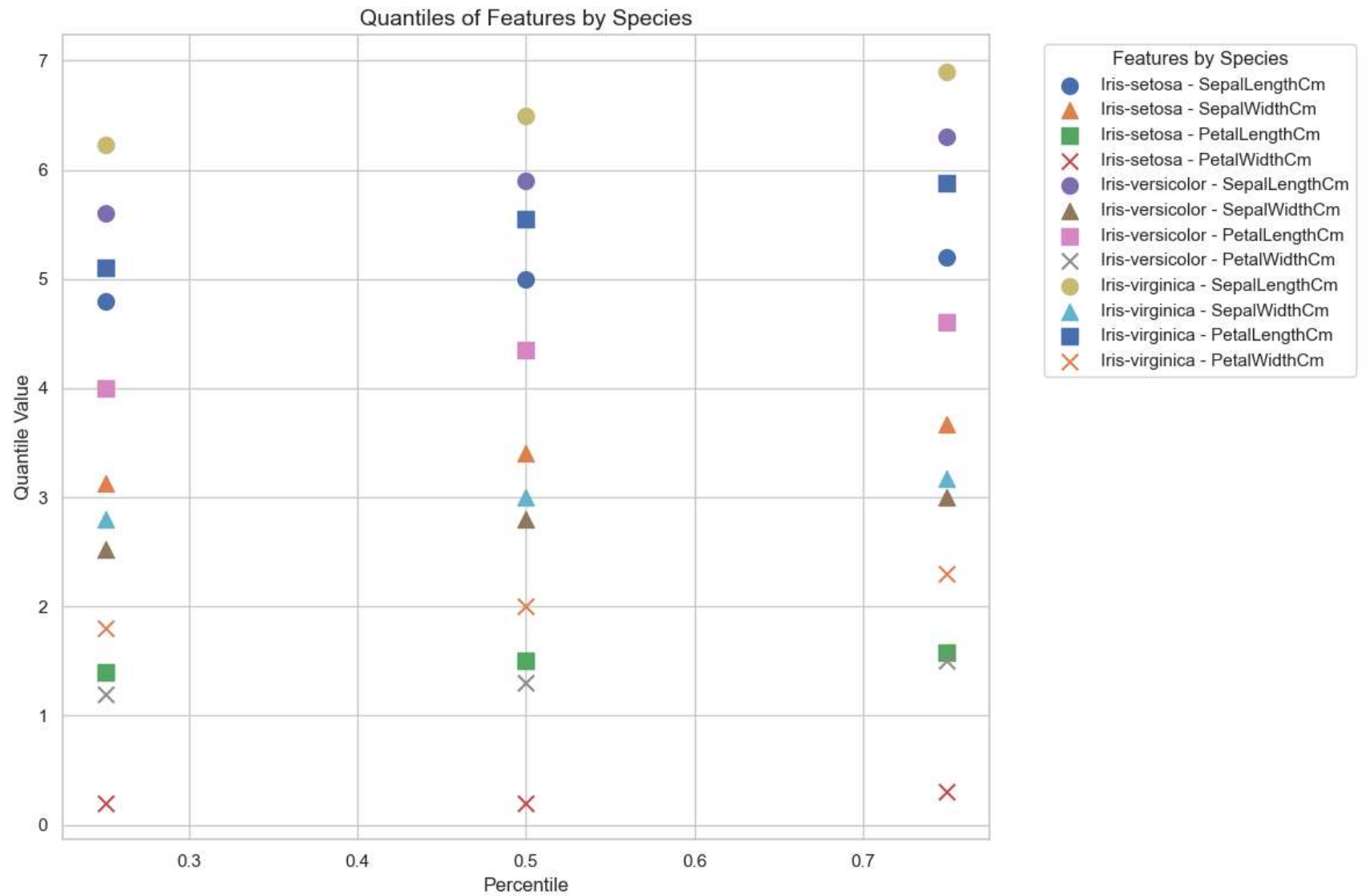
	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	50.00000	50.00000	50.00000	50.00000	50.00000
mean	75.50000	5.93600	2.77000	4.26000	1.32600
std	14.57738	0.51617	0.31379	0.46991	0.19775
min	51.00000	4.90000	2.00000	3.00000	1.00000
25%	63.25000	5.60000	2.52500	4.00000	1.20000
50%	75.50000	5.90000	2.80000	4.35000	1.30000
75%	87.75000	6.30000	3.00000	4.60000	1.50000
max	100.00000	7.00000	3.40000	5.10000	1.80000

Statistical Summary for Iris-virginica:

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	50.00000	50.00000	50.00000	50.00000	50.00000
mean	125.50000	6.58800	2.97400	5.55200	2.02600
std	14.57738	0.63588	0.32249	0.55189	0.27465
min	101.00000	4.90000	2.20000	4.50000	1.40000
25%	113.25000	6.22500	2.80000	5.10000	1.80000
50%	125.50000	6.50000	3.00000	5.55000	2.00000
75%	137.75000	6.90000	3.17500	5.87500	2.30000
max	150.00000	7.90000	3.80000	6.90000	2.50000

```
In [40]: import matplotlib.pyplot as plt
df1['SepalLengthCm'] = pd.to_numeric(df1['SepalLengthCm'], errors='coerce')
df1['SepalWidthCm'] = pd.to_numeric(df1['SepalWidthCm'], errors='coerce')
df1['PetalLengthCm'] = pd.to_numeric(df1['PetalLengthCm'], errors='coerce')
df1['PetalWidthCm'] = pd.to_numeric(df1['PetalWidthCm'], errors='coerce')
```

```
df1_clean = df1.dropna(subset=['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm'])
species_group = df1_clean.groupby('Species')
percentiles = [0.25, 0.50, 0.75]
plt.figure(figsize=(12, 8))
for i, (species, groupdata) in enumerate(species_group):
    quantilevalues = groupdata[['SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm']].quantile(percentiles)
    xvalue = percentiles
    plt.scatter(xvalue, quantilevalues['SepalLengthCm'], label=f'{species} - SepalLengthCm', marker='o', s=100)
    plt.scatter(xvalue, quantilevalues['SepalWidthCm'], label=f'{species} - SepalWidthCm', marker='^', s=100)
    plt.scatter(xvalue, quantilevalues['PetalLengthCm'], label=f'{species} - PetalLengthCm', marker='s', s=100)
    plt.scatter(xvalue, quantilevalues['PetalWidthCm'], label=f'{species} - PetalWidthCm', marker='x', s=100)
plt.title('Quantiles of Features by Species', fontsize=14)
plt.xlabel('Percentile', fontsize=12)
plt.ylabel('Quantile Value', fontsize=12)
plt.legend(title='Features by Species', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True)
plt.tight_layout()
plt.show()
```



In []:

In []:

In []:

In []: